Laboratory Project
Real-Time Systems
IST - 2025/2026

(Part 1 – PIC)
Weather Station and Alarm Clock

1 Introduction

Many real-time embedded systems are developed using relatively simple platforms, with low resources, where the utilization of operating systems to support the application may not be viable. In these type of systems there is essentially the need to do some elementary processing and access several input / output devices (e.g. access sensors to collect information). However, in several other cases, the existence of support at operating system level (even if with reduced functionality) may significantly facilitate the development of applications.

The laboratory projects to be implemented (composed of 2 parts) have as main goal the familiarization of students with the development of real-time embedded systems, both with medium / low complexity, using microcontrollers and without the support of an operating system, and also using multitasking kernels for the development of concurrent applications. In particular, they should acquire some expertise in the utilization of communication and synchronization mechanisms between tasks, in the context of concurrent applications, and should familiarize with other real-time embedded systems characteristics such as: access devices using simple digital input/output lines or network/buses such as I2C (Inter Integrated Circuit) or SPI (Serial Peripheral Interface); save information in non-volatile memory devices (EEPROM); utilization of analog-digital conversion; programming timers; use interrupts; etc.

They should also be aware of aspects related to energy consumption, resorting to operation modes that will allow as much as possible to increase system autonomy, but without compromising timing restrictions that may exist.

2 Problem description

The first part of the project corresponds to a "Weather Station and Alarm Clock" that has a rudimentary user interface (buttons/switches, LEDs, LCD) and no operating system support. The other part, with a more elaborated user interface and a higher processing capability, has a multitasking kernel to support a concurrent application.

This part of the project is implemented using the development board **"Curiosity High Pin Count Development Board"** [1], which includes a microcontroller **PIC16F18875** [2]. The application is programmed using the **C programming language** ("MPLAB XC8 C Compiler" [3]) and the development environment "MPLABX Integrated Development Environment (IDE)" [4, 5] from Microchip. Reading the related documentation is fundamental for correct project implementation.

The main functions to provide are essentially:

- Clock (to keep current time and to allow alarm management and timestamping of events).
- Periodic sensor reading (temperature and luminosity) with the possibility of storing data and handling alarm situations.
- Basic user interface.
- Incorporation of power-saving mechanisms.

The clock is built based on one of the timers available in the PIC (e.g. T1), and must provide the current time in the form of hours, minutes and seconds. Correct clock operation should be preserved whenever possible.

The sensors should be read in a periodic fashion (period `PMON`), and, depending on the collected values of temperature and luminosity, alarms may be generated, if enabled. The temperature sensor is a TC74 [7] (to be connected to the board), and the "luminosity sensor" is emulated using the potentiometer available in the developing board and converting the obtained value into 4 different levels {0,...,3}.

The collected sensor values may also be saved in non-volatile memory (PIC's internal EEPROM). They are saved as a record that, in addition to the values obtained from the sensors, also contains a timestamp with the current time. The size of the record is 5 bytes (h,m,s,T, L – corresponding to the timestamp (hours, minutes, seconds) and the values of temperature and luminosity). The records to save are the records corresponding to the maximum and the minimum values of temperature and luminosity (4 records, each with the 5 bytes explained above).

In what concerns alarms, there are 3 possible alarm situations: reaching the specified alarm clock time; temperature value **above** specified temperature threshold; luminosity level **below** specified luminosity threshold. If alarms are enabled any of the previous situations will imply an alarm notification: writing a letter on the LCD (C,T, or L, respectively); **and** generating a PWM signal. In this version of the system that PWM signal is applied to one LED (changing the brightness), but could be applied to a buzzer if available. The PWM signal will be active only for a duration of `TALA`, but the letter(s) will stay in the LCD until user interaction.

Relevant configuration parameters for the correct system operation, and other necessary operational values, should be preserved in non-volatile memory (PIC's internal EEPROM) so as to use the latest defined values if/when a restart operation is performed (reuse of saved values is done if they are in a consistent state (validated using a "magic word" and a checksum), otherwise the default values are used instead). Those parameters should be updated in the EEPROM when they are modified. The clock value should also be saved (only hours and minutes, seconds will be recovered as 0). Initial values for those parameters are:

PMON	5 sec	monitoring period
TALA	3 sec	duration of alarm signal (PWM)
TINA	10 sec	inactivity time
ALAF	0	alarm flag – alarms initially disabled
ALAH	12	alarm clock hours
ALAM	0	alarm clock minutes
ALAS	0	alarm clock seconds
ALAT	20 °C	threshold for temperature alarm
ALAL	2	threshold for luminosity level alarm
CLKH	0	clock hours
CLKM	0	clock minutes

3 User interface

The user interface, in this part of the project, is performed through the use of 2 buttons/switches (S1, S2 – connected to port pins RB4 and RC5, respectively), 4 LEDs (D2-D5 – connected to RA4-RA7), and an LCD (2 lines of 16 characters – to be connected to the board) [6]. The switches are used for setting the correct time of the clock, to define alarms, and to select operation mode. The LEDs and LCD are used to show the state of the system, and may have different meanings in normal mode and in configuration mode.

The information presented on the LEDs, in normal mode, will have the following meaning:

C	A	T	L
X	**X**	**X**	**X**
D5	D4	D3	D2
RA7	.	.	RA4

`C` (LED D5)	clock activity (blink – toggling every 1 second)
`A` (LED D4)	alarm signal (PWM signal with duration of TALA)
`T` (LED D3)	ON if temperature is above threshold; OFF otherwise
`L` (LED D2)	ON if luminosity is below threshold; OFF otherwise

The information presented on the LCD will have the following aspect:

hh:*mm*:*ss*	CTL AR
tt C	L *l*

In normal mode, current values of clock ("`hh:mm:ss`"), temperature ("`tt`") and luminosity level ("`l`") are presented. Letter "`A`" (or "`a`") will indicate if alarms are enabled (or disabled). Letters "`CTL`" will appear only in the case where the respective alarm (clock, temperature or luminosity) has occurred (they are kept until user interaction using switch S1).

In configuration mode, "`tt`" and "`l`" will present the current threshold values for temperature and luminosity alarms, and the letters "`CTL`" can be used as a place to select the modification of the alarm thresholds: clock, temperature and luminosity, respectively. When the cursor is on top of a given letter, the current specified alarm threshold value should be presented in the respective data field (clock, temperature or luminosity). If modification is selected (user presses S2), the cursor is placed on top of that data field, allowing the modification of that threshold. The letter "`a`" (or "`A`" – it works in "toggle" mode) shows if the alarms are disabled or enabled, and make it possible to enable/disable the alarms (all of them). The letter "`R`" (that only appears in configuration mode) can be used to reset maximum and minimum records, if desired (by pressing S2).

Switch S1 (RB4), in addition to clear possible alarm information (letters CTL), is used to enter configuration mode and move between modifiable fields. Switch S2 (RC5), in configuration mode, is used to increment the selected field or select the desired modification. More precisely, starting from normal operation mode, pressing S1 will make the system enter configuration mode allowing the modification of the first modifiable field (clock-hours). The cursor will be over that field, and it will be possible to increment the value of the field by pressing S2. Whenever S1 is pressed the cursor will move to the next field, until reaching normal operation mode again (clock-minutes, clock-seconds,

C,T,L, alarm-enable, reset max/min, normal mode again). When S2 is pressed, the value of the field that is being modified is incremented module the range of possible values for that field, or the given modification operation is selected (when on top of letters CTL).

Range values:

hh - hours [00 .. 23]

mm - minutes [00 .. 59]

ss - seconds [00 .. 59]

tt - temperature [00 .. 50]

l - luminosity level [0 .. 3]

a - alarm enabled/disabled [A,a]

Switch S2, in normal mode, can also be used to present the saved records corresponding to the maximum and minimum values. Those records are presented in sequence with each press of S2: first the maximum and minimum for temperature (one in each LCD line, as represented below); then luminosity; and then normal mode again. If the user does not press the full sequence of S2, after time TINA there should be an automatic return to normal mode.

hh:*mm*:*ss*	*tt* C	L	*l*
hh:*mm*:*ss*	*tt* C	L	*l*

The characteristics of the IO devices that are used should be consulted on the manual of the board [1], and/or on the "Data Sheets" of the devices.

4 Project development

In the development of the project, the utilization of a modular structure and a phased testing is advised.

In the target board there will be no operating system to support the execution of the application. It is students responsibility to structure the program in a modular fashion according to the several tasks that must be performed. The execution support to those tasks should be organized in the form of a "cyclic executive", resorting to the use of interrupts in the situations where that is justifiable. If needed, a state machine approach should also be used.

Underlying all aspects of project implementation, there should be the concern of, without jeopardizing the desired functionality, trying to optimize energy consumption, making the microcontroller enter a power saving mode whenever possible (using the instruction"`SLEEP()`"). Optionaly, the temperature sensor TC74 should also be put in standby mode between sensor readings.

The structure of the program should also be prepared to easily incorporate in future versions (if desired) USART serial communication (with baud rate 9600). Make sure that the associated timing requirements are satisfied.

5 Project delivery

This part of the project should be delivered by 07/12/2025. The delivery consists of a digital copy (ZIP file) of all developed programs and a small report (2 or 3 pages) explaining the main requirements and solutions related to real-time aspects and interaction between the main components. In addition to source files, the ZIP file should include final executable file (file "pic.hex"). All these elements should be correctly identified (group number and students). Project demonstrations (to be done later, together with part2) will be based on the delivered material.

References

[1] Microchip Technology Inc. *Curiosity High Pin Count Development Board User's Guide.* 2016-2018.

[2] Microchip Technology Inc. *PIC16(L)F18855/75 Data Sheet.* 2015-2018.

[3] Microchip Technology Inc. *MPLAB XC8 C Compiler User's Guide for PIC MCU.* 2022.

[4] Microchip Technology Inc. *MPLABX IDE User's Guide.* 2022.

[5] Microchip Technology Inc. *MPLAB Code Configurator v3.xx User's Guide.* 2022.

[6] Hitachi. *HD44780U (LCD-II) (Dot Matrix Liquid Crystal Display Controller/Driver).*

[7] Microchip Technology Inc. *TC74 - Tiny Serial Digital Thermal Sensor.* 2002.